FAST — NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES
FAST School of Computing · Karachi Campus · Department of Artificial Intelligence & Data Science
AL-2001 Programming for Artificial Intelligence (Laboratory) · Fall 2026

PROJECT PROPOSAL

Group Information

	Name	Roll Number	Section	Role in the project
Leader	Ayesha Nehal	25K0002	BSAI-3A	Class hierarchy, model training and comparison, console menu, system integration, repository setup
Member 2	M.Hamdan	25K0136	BSAI-3A	Text preprocessing, NumPy features, visualisations
Member 3	Mohsina Zeeshan	25K0095	BSAI-3A	Data layer, API and cache, Pandas analysis

Project Title	**TweetPulse: Reading the Mood of Twitter with NLP**
Specialisation Track	Track A — Text / NLP ☒ Track B — Images / CV ☐
GitHub Repository URL	https://github.com/AyeshaNehal/TweetPulse
Date of Submission	19th October 2026

1. Project Title and Summary

State the title of your project, then summarise it in four to six sentences: what the software does, who would use it, and what the user gets out of it. Write it so that a reader who knows nothing about your topic understands the idea by the end of the paragraph. Avoid listing libraries here — that belongs in Section 6.

TweetPulse: Reading the Mood of Twitter with NLP

TweetPulse is a console application that decides whether a short social-media post is positive, negative or neutral. The user can explore a collection of about 27,000 labelled tweets, type in any sentence of their own, or fetch fresh headlines from the web and see how the system classifies them, along with a confidence score. Behind the menu, the program cleans messy text, converts it into numbers and trains several models, then compares them against a ready-made rule-based scorer to show which approach is more reliable and where each one is fooled by sarcasm, slang or negation. Students, small businesses and social-media analysts could use it to understand public opinion without reading thousands of posts by hand. The result is a clear set of tables and charts that explain not only what the sentiment is, but how trustworthy each method is.

2. Problem Statement and Motivation

Describe the problem your project addresses in 120–180 words. What is difficult, slow or error-prone without your software? Who currently faces this, and how do they cope today? Finish with one sentence on why solving it is worthwhile. Be specific — "managing data is hard" says nothing; "a lab instructor recalculating weighted averages for 40 students across 5 subjects by hand each week" says a great deal.

Every day, millions of tweets express opinions about products, services, events and public figures, but nobody can read them all. A small business owner checking customer reactions to a new product, or a student analysing public reaction to an event, currently has to scroll through posts and judge each one by hand, which takes hours for a few hundred tweets and gives inconsistent results because two readers often disagree.

Keyword searches do not help much either, because tweets are full of slang, hashtags, @mentions, web links, stretched words such as 'soooo good' and negation such as 'not bad', which fool simple word matching. Existing sentiment tools are also usually black boxes: they give an answer without showing how reliable it is or where it fails. Our project provides a transparent system that classifies tweets automatically, measures its accuracy honestly on unseen data, and shows exactly which kinds of tweets confuse it. Solving this is worthwhile because it turns an unmanageable pile of text into a trustworthy, explainable summary.

3. Proposed Solution

Explain how your program will solve the problem, in 150–200 words. Walk through what happens from the user's point of view: what they provide, what the system does with it, and what it gives back. Then name the main components you expect to build — for example a catalogue loader, an analysis engine and a reporting module — and say in one line what each is responsible for.

The user starts the program and sees a numbered console menu. They load the tweet dataset, which is cleaned and summarised, then explore it, watch a single tweet being broken into tagged words, and score tweets with a rule-based baseline. The training option splits the data once into training and test portions, converts the cleaned text into numerical features, trains the models and prints a comparison of accuracy, precision, recall and F1, followed by charts. The user can inspect tweets where the models disagree, type their own text for a prediction with confidence, or fetch live headlines from a public web API, cached so the program still works offline. Invalid input, missing files and network failures produce clear messages, not crashes.

Main components:

- Data loader: reads CSV and JSON, calls the API, caches replies, raises DataLoadError.
- Preprocessor: our own text-cleaning and tokenising code.
- Feature module: builds the NumPy Bag-of-Words matrix and word statistics.
- Analyser: Pandas cleaning, filtering and grouping.
- Models and evaluator: classifier classes, training and metrics.
- Visualiser: saves five labelled figures.
- Menu: validated input and navigation.

3.1 Planned Class Design

List the classes you intend to write, and for each give its main attributes and methods and its relationship to the others. A simple indented list is acceptable at proposal stage; the full diagram belongs in the report.

- Tweet — one tweet and its label
  - Attributes: _text, _label, _tokens (private, exposed through properties)
  - Methods: constructor that rejects empty text, clean(), __str__()
  - Relationship: created by the data loader and passed to the classifiers

- SentimentClassifier (abstract base class)
  - Attributes: _name, _is_trained (encapsulated)
  - Methods: abstract train(texts, labels) and predict(text) returning a (label, confidence) tuple; concrete predict_many(texts)
  - Relationship: parent of the three classifiers below
- VaderClassifier(SentimentClassifier) — rule-based baseline; overrides train() (no training needed) and predict() by mapping the compound score to a label
- NaiveBayesClassifier(SentimentClassifier) — constructor takes the feature type (Bag-of-Words or TF-IDF); overrides train() and predict()
- LogisticClassifier(SentimentClassifier) — constructor takes the n-gram range (unigrams or unigrams plus bigrams); overrides train() and predict()
- DataLoadError(Exception) — custom exception raised by the data loader
- Polymorphism: the evaluator loops over a list of SentimentClassifier objects and calls train() and predict() on each without knowing its type.

4. Scope

In scope — the features you commit to delivering. List the features that must work for the project to be considered complete.

#	Feature	Why it is essential
1	Load train.csv, clean missing values and duplicates, and summarise by sentiment, country and age group	Gives every later step reliable data and covers the Pandas requirement
2	Our own text-cleaning pipeline followed by Bag-of-Words and TF-IDF features	Core of the NLP track; models cannot learn from raw tweets
3	VADER baseline plus Naive Bayes and Logistic Regression, compared on one shared train/test split	Provides the honest evaluation and model comparison
4	Five labelled visualisations, a NumPy word-count matrix and a disagreement analysis of tweets the models get wrong	Turns the numbers into findings and covers the NumPy and Matplotlib requirements
5	Live API fetch with JSON cache and offline fallback, plus classification of user-typed text	Meets the web-API requirement and makes the program interactive and robust

Out of scope — deliberately excluded. Naming what you will not build is as important as naming what you will; it is how an examiner judges whether your scope is realistic.

A graphical or web interface; a database server; deep learning and pretrained transformer models; deployment; scraping live Twitter data; languages other than English; and detecting sarcasm automatically (we only analyse it in the failure cases).

5. Data Source

Identify exactly where your data will come from, and confirm that you can actually obtain it. A project whose data source turns out to be unreachable in week 12 cannot be rescued.

Source name and URL	Sentiment Analysis Dataset (train.csv), Kaggle: https://www.kaggle.com/datasets/abhi8923shriv/sentiment-analysis-dataset?select=train.csv Live data for the web-API requirement: Hacker News public API, https://hacker-news.firebaseio.com/v0/ (story headlines). Files used from the Kaggle page: train.csv and test.csv only; its other files, including a 1.6-million-row file, are not used.
Type	Public web API ☒ Downloadable dataset ☒ Self-collected / generated ☐
Access requirements	No key needed ☒ Free key required ☐ (Paid sources are not permitted) ☒ No key needed (the API needs no key; train.csv is downloaded once from Kaggle and stored in data/raw/, so no key is needed at run time)
Approximate size	About 27,000 tweets, around 10 columns (textID, text, selected_text, sentiment, Time of Tweet, Age of User, Country and country statistics), a few MB ,test.csv is 474 kB with 9 columns (no selected_text). API replies are a few KB per request.
Licence / terms of use	CC0: Public Domain
Offline fallback	Every successful API reply is saved to data/raw/api_cache.json; when the network is down or the status code is not 200, the program loads this cache and says so. The dataset is read from the local data/raw/train.csv (or a small committed sample if the full file cannot be included), and a missing or malformed file raises DataLoadError with clear instructions instead of crashing.

> *Required: your program must still run when the network is unavailable, by reading a locally cached copy of the data. Describe your caching plan in the last row above.*

6. Technologies and Lab Coverage

For each row, state in one line how your project will use that concept. If you genuinely cannot see a use for one, write "to be identified" — but every row must be satisfied by the time the report is written.

Lab	Concept to be applied	How your project will use it
01–03	Core Python, control flow, data structures	Validated menu loop with int() conversion and re-prompting, branching, loops and comprehensions for token filtering. List for tweet tokens, tuple for (label, confidence), dict for word frequencies and label mapping, set for stopwords and vocabulary; each choice justified in the report.
04	Functions, lambda, map / filter	Documented functions with defaults such as clean_text(text, lowercase=True, remove_stopwords=True); lambda with sorted(key=...) for top words, and map / filter in preprocessing.
05	Classes, inheritance, encapsulation, polymorphism	Abstract SentimentClassifier with three overriding subclasses, a Tweet class with private attributes and properties, and polymorphic calls over a list of classifiers.
06	Files (TXT / CSV / JSON), exceptions, web API	CSV for the dataset and saved predictions, JSON for the API cache and metrics, TXT for a run log. Custom DataLoadError and specific except blocks (FileNotFoundError, json.JSONDecodeError, requests connection errors and timeouts). Live headlines via requests with the status code checked and the reply cached.
07	NumPy arrays, broadcasting, vectorisation	2-D Bag-of-Words matrix (about 2,000 tweets × 500 words), boolean masking per sentiment class, row normalisation by broadcasting, and sum(axis=0) for top words per class, all without element loops.
08	Pandas — loading, cleaning, grouping	DataFrame from train.csv; dropna() and drop_duplicates(); filtering by country or age group; groupby('sentiment') for counts, average tweet length and average VADER score.
09	Matplotlib / Seaborn visualisations	Five labelled figures: class counts, tweet-length histogram by class, VADER compound score per label, confusion-matrix heatmap, and model F1 comparison; each interpreted in the report.
10–11	Machine learning, model comparison	One stratified train_test_split (fixed random_state) shared by all models. Compare VADER, Naive Bayes + BoW, Naive Bayes + TF-IDF, Logistic Regression + TF-IDF, and Logistic Regression + TF-IDF with bigrams, using accuracy, precision, recall, F1 and a confusion matrix on the test set only. The best model is finally scored once on the labelled test.csv, which is never used for fitting or tuning.

Lab	Concept to be applied	How your project will use it
12 or 13	Chosen track — NLP text features or image preprocessing (Track A)	Our own cleaner (lowercasing, regex removal of URLs, @mentions, # symbols, punctuation and digits, collapsing repeated letters, tokenising, stopword removal) followed by CountVectorizer and TfidfVectorizer features. NLTK supplies only the stopword list and VADER, and is cited.
14	Documentation, modularity, ethical considerations	Modular src/ layout, docstrings throughout, README, pinned requirements.txt, and an ethics discussion covering tweet privacy, demographic bias, sarcasm and slang, and misuse for profiling.

7. Division of Work

State clearly which member is responsible for which part. This table will be compared against your GitHub commit history and against the contribution table in the final report, so make it honest and keep it updated.

Member	Modules / components owned	Approximate share
Leader	models.py (abstract base class, three classifiers, Tweet class), evaluate.py (stratified split, metrics, model comparison, final test.csv check), main.py (menu, input validation, integration of all modules), repository setup, README and final integration	33.33%
Member 2	preprocessing.py (own cleaner, data structures, lambdas), numpy_ops.py (Bag-of-Words matrix), visualize.py (five figures	33.33%
Member 3	data_loader.py, exceptions.py (CSV/JSON/TXT, API, cache, custom exception), analysis.py (Pandas)	33.33%

8. Milestones and Timeline

Break the work into weekly or fortnightly milestones from now until the submission deadline, which falls one week before the Lab Final Exam. Each milestone should name something demonstrable, not an activity — "catalogue loads from JSON and the menu runs" rather than "work on the data layer".

#	Milestone — what will be demonstrably working	Target date	Owner
1	Repository and folder skeleton pushed with a first commit from every member; train.csv loads and the menu starts and exits cleanly	Week 1	All
2	Cleaned DataFrame summarised in the menu; API reply saved to cache and reloaded with the network off; missing-file error handled by DataLoadError; our cleaner turns a raw tweet into tokens	Week 2	Member 2, 3

#	Milestone — what will be demonstrably working	Target date	Owner
3	Bag-of-Words matrix shapes printed; the abstract class and three classifiers train; comparison table of all five configurations prints on one shared split	Week 3	Member 2, Leader
4	Five figures saved to outputs/figures/; full menu works including disagreement analysis and live-text prediction; program survives a missing file, malformed row and no network	Week 4	All
5	Final integration, documentation and repository cleanup: README, requirements.txt, proposal and report as PDF and MD in docs/, and a successful run from a fresh clone on another machine	Week 5	All

9. Expected Outcome and Risks

In 80–120 words, describe what a working submission will look like — what the examiner will see when they run it, and what results you expect to be able to report. Then name the two risks most likely to derail the project (for example, the API rate-limiting you, or the dataset proving too small to train on) and state what you will do if each occurs.

When the examiner runs the program from a fresh clone, a numbered menu appears. Loading the dataset reports its shape and cleaning results; training prints a table comparing five configurations on the same unseen test split; the figures folder fills with five labelled charts; and typing a tweet returns a label with confidence. We expect the trained models to beat or match the rule-based baseline, with accuracy of roughly 65–75% on three classes and the neutral class being the hardest, and we will report this honestly with a confusion matrix and examples of failures.

Risk 1: the dataset cannot be included or downloaded on a clean clone (licence, size, or Kaggle access). Response: commit a small stratified sample, give the download link and exact placement steps in the README, and have the loader detect a missing file and explain what to do.
Risk 2: the live API is unreachable or changes its format. Response: keep a committed cached reply that the program falls back to, validate the response fields before using them, and test the whole program with the network disabled.

For office use — Lab Instructor	
Proposal status	Approved ☐ Approved with changes ☐ Revision required ☐
Comments	
Signature and date	

End of Project Proposal